

Aplicație de calcul pentru configurarea unui robot 3D

Tipuriță Alexandru

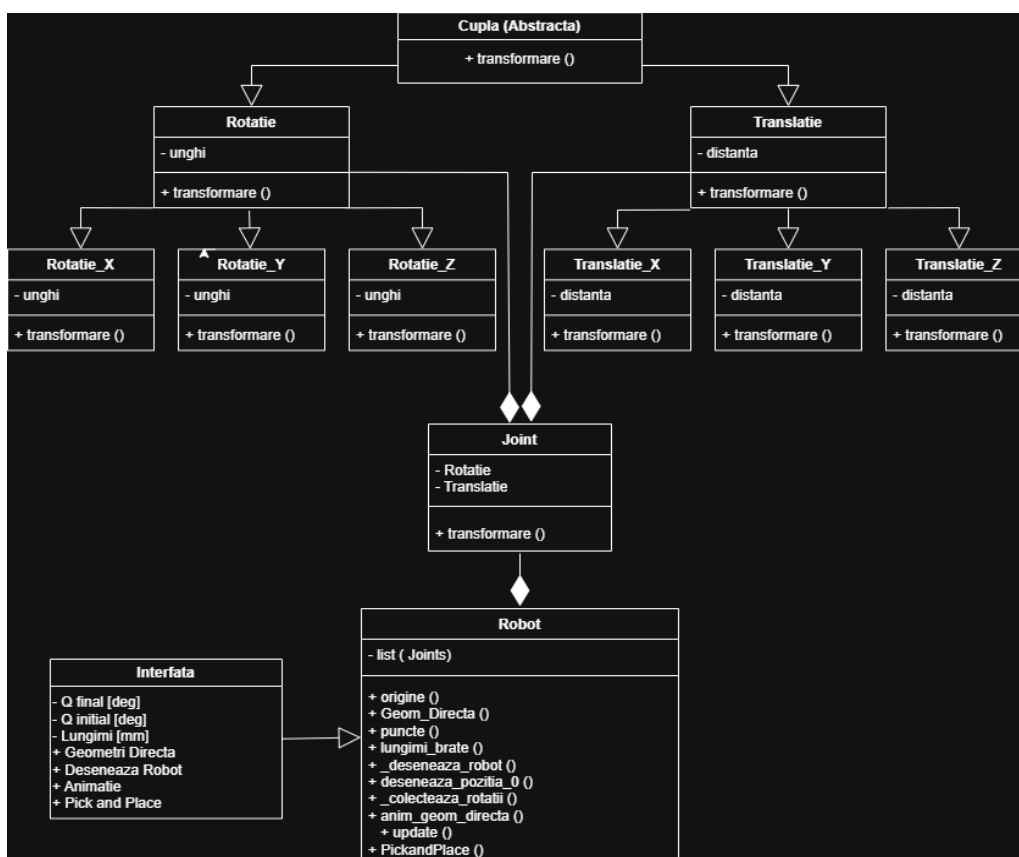
Condrea Laurențiu

Palamiuc Ilie

Introducere

Prezenta documentație tehnică descrie dezvoltarea unei aplicații software pentru configurarea și simularea unui robot serial, realizată în limbajul Python pe baza principiilor programării orientate pe obiecte. Aplicația a fost concepută modular, utilizând concepte precum abstractizarea, moștenirea și compoziția, pentru a asigura o structură flexibilă și ușor de extins.

Scopul aplicației este modelarea mișcărilor cinematice ale robotului, generarea configurațiilor geometrice și simularea funcționării acestuia în spațiul tridimensional. Sistemul permite definirea diferitelor tipuri de rotații și translații, gestionarea articulațiilor robotice și realizarea animațiilor specifice proceselor de manipulare de tip Pick and Place.



Tematică

Pentru elaborarea modelului geometric al manipulatorului se consideră următoarele ipoteze simplificatoare:

- Configurația robotului este analizată static, independent de timp.
- Mișcarea nu este luată în considerare, fiind neglijate vitezele liniare și unghiulare.
- Nu se consideră influențe externe sau interne, precum gravitația ori momentele actuatorilor.
- Parametrii geometrici ai structurii sunt considerați perfect cunoscuți, fără erori sau incertitudini.

Parametrii principali ai manipulatorului sunt reprezentați de numărul cuplelor, tipul fiecărei cuple, geometria elementelor componente (lungimile brațelor), precum și pozițiile unghiulare ale cuplelor.

$$\mathbf{R}_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(q_i) & -\sin(q_i) \\ 0 & \sin(q_i) & \cos(q_i) \end{bmatrix} T_x = \begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$\mathbf{R}_y = \begin{bmatrix} \cos(q_i) & 0 & \sin(q_i) \\ 0 & 1 & 0 \\ -\sin(q_i) & 0 & \cos(q_i) \end{bmatrix} T_y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$\mathbf{R}_z = \begin{bmatrix} \cos(q_i) & -\sin(q_i) & 0 \\ \sin(q_i) & \cos(q_i) & 0 \\ 0 & 0 & 1 \end{bmatrix} T_z = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Geometria directă determină poziția efectorului final al robotului pe baza parametrilor articulațiilor (unghiuri sau translații). Pentru un manipulator serial, poziția finală se obține prin înmulțirea succesivă a matricilor de transformare asociate fiecărei articulații și fiecărui segment al brațului.

Distanța dintre baza fixă și prima cuplă nu a fost inclusă în model, deoarece analiza cinematică începe direct de la prima articulație activă. Astfel, transformările sunt definite succesiv, de la o cuplă la alta.

Punctele $p_0 \dots p_n$ reprezintă pozițiile articulațiilor și ale efectorului final în raport cu sistemul de coordonate global. Acestea se obțin prin aplicarea matricilor de transformare asupra vectorului inițial omogen $P_0 = [0; 0; 0; 1]$.

Pentru reprezentare se folosesc punctele din geometria directă, care definesc segmentele dintre elemente și permit vizualizarea structurii robotului.

Sarcina pick and place constă în deplasarea brațului robotic pentru preluarea unui obiect dintr-o poziție inițială și plasarea lui într-o poziție finală. Din punct de vedere al traiectoriilor, aceasta reprezintă mișcarea efectorului final între cele două puncte.

Traectoria dintre cele două poziții nu este impusă. Pentru control în spațiul cartezian se pot introduce puncte intermediare, formând traiectorii elementare între poziții succesive.

Interpolarea polinomială este utilizată pentru generarea traiectoriilor în spațiul articular al roboților, prin definirea unei funcții de forma:

$$q(t) = a_3 \cdot t^3 + a_2 \cdot t^2 + a_1 \cdot t + a_0$$

unde $q(t)$ reprezintă poziția în funcție de timp, iar coeficienții a_0 , a_1 , a_2 , a_3 sunt determinați pentru a respecta condițiile cinematice și de continuitate.

$$\left(a_3 = -2 \frac{q_\Delta - q_0}{\Delta^3} \right),$$

$$\left(a_2 = 3 \frac{q_\Delta - q_0}{\Delta^2} \right),$$

$$(a_1 = 0),$$

$$(a_0 = q_0),$$

unde (q_Δ) reprezintă poziția finală, iar (q_0) reprezintă poziția inițială, iar (Δ) este intervalul de timp pe care se realizează tranziția.

Arhitectura Software – Classes.py

Biblioteca Numpy:

Biblioteca Numpy este utilizată pentru realizarea operațiilor matematice și manipularea matricilor necesare transformărilor geometrice tridimensionale.

Funcții principale utilizate:

- **np.array()** – creare matrici
- **np.cos()** – calcul cosinus
- **np.sin()** – calcul sinus
- **np.eye()** – generare matrice identitate
- **np.round()** – rotunjirea valorilor numerice

Modulul ABC

Modulul abc este utilizat pentru implementarea claselor abstracte, în cazul de față o interfață comună pentru toate tipurile de transformări.

Structura fișierului Classes.py

Clasă	Rol
Cupla	Interfață pentru transformări
Rotatie_X, Rotatie_Y, Rotatie_Z	Transformări de rotație
Translatie_X, Translatie_Y, Translatie_Z	Transformări de translație
Joint	Compunerea matricei de rotație-translație

Clasa abstractă - Cupla

Clasa Cupla reprezintă clasa de bază pentru toate transformările geometrice. Metoda transformare(self) trebuie implementată de toate clasele derivate și are rolul de a returna matricea asociată transformării respective.

Cod:

```
class Cupla(ABC):  
  
    @abstractmethod  
    def transformare(self):  
        pass
```

Clasele de rotație

Clasele de rotație primesc ca argument un unghi dorit de utilizator, iar apoi implementează o rotație în jurul axelor sistemului cartezian. Aceste rotații sunt definite sub formă matricială, utilizând matrici de transformare de dimensiune 4×4 , ceea ce permite

reprezentarea matematică a orientării și compunerea ușoară a mai multor transformări geometrice.

Clasa Rotatie_X

Parametru – unghi (grade)

Cod:

```
class Rotatie_X(Cupla):

    def __init__(self, unghi):
        self.unghi = unghi

    def transformare(self):

        c = np.cos(self.unghi)
        s = np.sin(self.unghi)

        M = np.array([
            [1, 0, 0, 0],
            [0, c, -s, 0],
            [0, s, c, 0],
            [0, 0, 0, 1]
        ])

        return np.round(M, 10)
```

Clasa Rotatie_Y

Parametru – unghi (grade)

Cod:

```
class Rotatie_Y(Cupla):

    def __init__(self, unghi):
        self.unghi = unghi

    def transformare(self):

        c = np.cos(self.unghi)
        s = np.sin(self.unghi)

        M = np.array([
            [c, 0, s, 0],
            [0, 1, 0, 0],
            [-s, 0, c, 0],
            [0, 0, 0, 1]
        ])
```

```
])  
  
return np.round(M, 10)
```

Clasa Rotatie_Z

Parametru – unghi (grade)

Cod:

```
class Rotatie_Z(Cupla):  
  
    def __init__(self, unghi):  
        self.unghi = unghi  
  
    def transformare(self):  
  
        c = np.cos(self.unghi)  
        s = np.sin(self.unghi)  
  
        M = np.array([  
            [c, -s, 0, 0],  
            [s, c, 0, 0],  
            [0, 0, 1, 0],  
            [0, 0, 0, 1]  
        ])  
  
        return np.round(M, 10)
```

Clasele de translație

Clasele de translație primesc ca argument o distanță introdusă de utilizator, iar apoi implementează o deplasare de-a lungul axelor sistemului cartezian.

Clasa Translatie_X

Parametru – distanță (mm)

Cod:

```
class Translatie_X(Cupla):  
  
    def __init__(self, distanta):  
        self.distanta = distanta  
  
    def transformare(self):  
  
        T = np.array([  
            [1, 0, 0, self.distanta],  
            [0, 1, 0, 0],  
            [0, 0, 1, 0],  
            [0, 0, 0, 1]  
        ])
```

```

        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ])

    return np.round(T, 10)

```

Clasa Translație Y

Parametru - distanță (mm)

Cod:

```

class Translatie_Y(Cupla):

    def __init__(self, distanta):
        self.distanta = distanta

    def transformare(self):

        T = np.array([
            [1, 0, 0, 0],
            [0, 1, 0, self.distanta],
            [0, 0, 1, 0],
            [0, 0, 0, 1]
        ])

        return np.round(T, 10)

```

Clasa Translație Z

Parametru - distanță (mm)

Cod:

```

class Translatie_Z(Cupla):

    def __init__(self, distanta):
        self.distanta = distanta

    def transformare(self):

        T = np.array([
            [1, 0, 0, 0],
            [0, 1, 0, 0],
            [0, 0, 1, self.distanta],
            [0, 0, 0, 1]
        ])

        return np.round(T, 10)

```

Clasa Joint

Clasa **Joint** are rolul de a compune matricile de rotație și translație, pornind de la o matrice identitate de dimensiune 4x4, pentru a modela o componentă a robotului.

Parametri – lista alcătuită din instanțe ale claselelor de rotație, respectiv translație

Cod:

```
class Joint:

    def __init__(self, transforms):
        self.transforms = transforms

    def transformare(self):

        T = np.eye(4)

        for t in self.transforms:
            T = T @ t.transformare()

        return np.round(T, 10)
```


Arhitectura Software – Robot.py

Clasa Robot

Constructorul __init

Metoda inițializează obiectul clasei și primește ca parametru joints, o listă de articulații (instanțe de tip Joint). Fiecare element din listă conține transformări geometrice (rotații și translații) care definesc structura cinematică a robotului.

Cod:

```
def __init__(self, joints):  
    self.joints = joints
```

Metoda origine

Metoda returnează originea sistemului de coordonate global, exprimată ca vector tridimensional. Ea reprezintă punctul de referință al sistemului cinematic al robotului.

Cod:

```
def origine(self):  
  
    return np.array([0.0, 0.0, 0.0])
```

Metoda Geom_Directa

Această metodă calculează geometria directă a robotului, adică determinarea punctului final pe baza transformărilor definite în articulații.

Inițializare:

- **M** - matricea de transformare inițială, setată ca matrice identitate 4x4
- **p_list** - este o listă care va stoca transformările intermediare ale robotului (matrici)

Pentru fiecare joint se execută următoarele operații:

- se înmulțește matricea curentă cu transformarea articulației
- se actualizează transformarea globală a sistemului
- se aplică rotunjirea numerică pentru stabilitate numerică
- se salvează o copie a transformării curente, reprezentând starea robotului după fiecare articulație

Metoda returneaza:

- **M** - transformarea finală a robotului
- **p_list** - lista transformărilor intermediare pentru fiecare articulație

Cod:

```
def Geom_Directa(self):

    M = np.eye(4)
    p_list = []

    for joint in self.joints:
        M = M @ joint.transformare()
        M = np.round(M, 10)
        p_list.append(M.copy())

    return M, p_list
```

Desen și animație

Librăria Matplotlib

Atât pentru desen cât și pentru animație a fost folosită librăria Matplotlib pentru randarea grafică 3D.

Componente folosite:

- `plt.figure(figsize = (8, 8))` — creează fereastra grafică;
- `fig.add_subplot(111, projection='3d')` — sistem de axe 3D (proiecție tridimensională);
- `ax.plot` — desenarea liniilor între cuple (scheletul robotului);
- `ax.quiver` — desenarea **săgeților 3D** pentru axele locale X/Y/Z la fiecare cuplă (convenția RGB ↔ XYZ);
- `ax.text` — etichete pentru identificarea cuplelor (C0, C1, ..., C6);
- `ax.set_xlim / ylim / zlim` — fixarea scalei (critic pentru animație, previne „pulsarea” graficului);
- `ax.clear` — ștergerea axelor înainte de redesenare în animație;
- `plt.show` — afișare cu event loop blocant.

Matplotlib Animation (*matplotlib.animation.FuncAnimation*)

Generarea animației prin apelarea repetată a unei funcții de update.

Mecanism:

- Primește o figură și o funcție `update(frame)`;
- Apelează `update` o dată la interval milisecunde;
- La fiecare apel, funcția modifică starea robotului și redesenează;

- Parametrul repeat = False oprește animația după ultima cuplă;
- Parametrul blit = False dezactivează optimizarea blit (incompatibilă cu 3D).

Desen geometrie directă

Reprezintă robotul în starea configurată prin:

- Linie neagră ce unește toate punctele cuplelor (scheletul brațelor);
- Markere circulare la fiecare cuplă;
- Triadă colorată (X roșu, Y verde, Z albastru) la fiecare cuplă, indicând orientarea cadrului local;
- Etichete C0...C6 pentru identificarea cuplelor;
- Limite calculate automat pe baza extinderii maxime a robotului.

Animația geometriei directe — anim geom directa

Vizualizarea dinamică a mișcării robotului de la poziția inițială către configurația țintă, secvențial, cuplă cu cuplă.

Algoritmul:

Faza de pregătire:

- Se colectează toate rotațiile din robot, salvând perechi [referință, unghi_țintă];
- Se calculează limita axelor ca **suma lungimilor brațelor** (raza maximă).

Faza de animație:

Pentru fiecare frame se determină:

- $\text{cupla_idx} = \text{frame} // \text{frames_per_cupla}$ — cupla activă;
- $\text{progres} = (\text{frame} \% \text{frames_per_cupla} + 1) / \text{frames_per_cupla}$ — fracția mișcării ($0 \rightarrow 1$);

Pentru fiecare cuplă:

- Cuplele anterioare $\rightarrow$ unghi fix la țintă;
- Cupla curentă $\rightarrow$ unghi interpolat liniar: $\text{țintă} \times \text{progres}$;
- Cuplele viitoare $\rightarrow$ unghi 0.

Se șterge graficul (ax.clear) și se redesenează cu noile valori.

Titlul dinamic afișează cupla activă, procentul de progres și unghiul curent în grade — feedback vizual pentru utilizator.

Pick and Place

Funcția PickandPlace() generează o traiectorie între două configurații ale robotului:

-poziția inițială $\rightarrow Q_initial$

-poziția finală $\rightarrow Q_final$

folosind o interpolare polinomială de ordinul 3.

Robotul pleacă dintr-o poziție și ajunge în alta, și se calculează:

-poziția în timp

-viteza

-acclerația

pentru fiecare articulație, într-un timp delta $\rightarrow$ se definește intern (default este pe 4 secunde).

Se transformă în vectori numpy din vectorii inițiali (liste):

```
Q_initial = np.array(Q_initial)
```

```
Q_final = np.array(Q_final)
```

Vectorul de timp generează 100 de puncte între 0 și delta:

```
t = np.linspace(0, delta, 100)
```

Folosim reshape pentru a transforma vectorul din (100,) în (1,100) deoarece calculele se fac simultan pentru toate articulațiile:

```
t = t.reshape(1, -1)
```

Coeficienții polinomului:

```
a0 = Q_initial.reshape(-1, 1)
```

```
a1 = np.zeros_like(a0)
```

```
a2 = 3 * (Q_final.reshape(-1, 1) - a0) / (delta ** 2)
```

```
a3 = -2 * (Q_final.reshape(-1, 1) - a0) / (delta ** 3)
```

Calculul poziției:

```
qt = a3 * t ** 3 + a2 * t ** 2 + a1 * t + a0
```

Calculul vitezei:

```
viteza = 3 * a3 * t ** 2 + 2 * a2 * t + a1
```

Calculul accelerației:

```
acceleratie = 6 * a3 * t + 2 * a2
```

Crearea figurii se realizează cu:

```
plt.figure(figsize=(10, 8))
```

```
plt.subplot(3,1,1)
```

Adică 3 grafice pe verticală: poziție, viteză și accelerație, apoi se iterează prin toate articulațiile:

```
for i in range(qt.shape[0]):
```

qt.shape fiind 5 în cazul nostru.

Și plotăm cu ajutorul:

```
plt.plot(t.flatten(), qt[i], label=f'q{i + 1}')
```

```
plt.grid(True)
```

```
plt.legend()
```

plt.tight_layout() - aranjează automat graficele să nu se suprapună.

Și se afișează graficele prin comanda:

```
plt.show()
```

Funcția returnând:

```
return qt, viteza, acceleratie, t.flatten()
```

Interfața

Partea de interfață este făcută cu biblioteca Tkinter și are rolul de a crea fereastra aplicației, câmpurile de introducere, butoanele și zona unde se afișează rezultatele.

Se creează fereastra principală a aplicației:

```
window = tk.Tk()
```

Setează titlul ferestrei:

```
window.title("Robot Simulator")
```

Dimensiunea ferestrei:

```
-1100 px lățime
```

```
-800 px înălțime
```

```
window.geometry("1100x800")
```

Setează culoarea de fundal:

```
window.configure(bg="#1e1e1e")
```

ttk.Style() permite modificarea aspectului widgeturilor, iar clam este o temă vizuală.

```
style = ttk.Style()
```

```
    style.theme_use("clam")
```

Configurează toate butoanele ttk.Button:

```
    -font Arial
```

```
    -mărime 11
```

```
    -padding interior 10 px
```

```
-style.configure("TButton", font=("Arial", 11), padding=10)
```

Titlul aplicației:

```
tk.Label(
```

```
    window,
```

```
    text="ROBOT SIMULATOR",
```

```
    font=("Arial", 24, "bold"),
```

```
    bg="#1e1e1e",
```

```
    fg="white"
```

```
    ).pack(pady=15)
```

Frame-ul pentru input-uri:

```
frame_input = tk.Frame(window, bg="#2b2b2b", padx=20, pady=20)
```

Poziționare input-uri:

```
frame_input.pack(side=tk.LEFT, fill=tk.Y, padx=20, pady=20)
```

Stiluri reutilizabile:

```
label_style = {"font": ("Arial", 11), "bg": "#2b2b2b", "fg": "white"}
```

```
    entry_style = {"font": ("Arial", 11), "width": 10}
```

Label-uri:

```
tk.Label(frame_input, text="q1", **label_style)
```

Entry-uri:

```
entry_q1 = tk.Entry(frame_input, **entry_style)
```

Organizarea elementelor ca într-un tabel se realizează cu:

```
.grid(row=, column=)
```

Valorile default:

```
entry_q1.insert(0, "180")
```

Unde:

-0 este index - începutul câmpului

-180 – text-ul inserat

Comanda butoanelor:

```
ttk.Button(  
    frame_buttons,  
    text="Geometrie Directa",  
    command=geometrie_directa  
)
```

Output-ul:

```
frame_rezultat = tk.Frame(window, bg="#252526", padx=20, pady=20)
```

Pentru ștergerea output-ului:

```
rezultat_text.delete(1.0, tk.END)
```

Pornirea aplicației grafice:

```
window.mainloop()
```

În interfața, excepțiile sunt folosite pentru a preveni blocarea aplicației atunci când utilizatorul introduce date greșite.

try:

```
#cod periculos
```

```
except ValueError:
```

```
#executabil dacă apare eroarea
```

O excepție apare când programul întâlnește o eroare în timpul execuției.

- utilizatorul scrie litere în loc de numere

Se execută acest cod când utilizatorul introduce litere în loc de numere:

```
rezultat_text.delete(1.0, tk.END)

rezultat_text.insert(

    tk.END,

    "Introdu valori valide!"

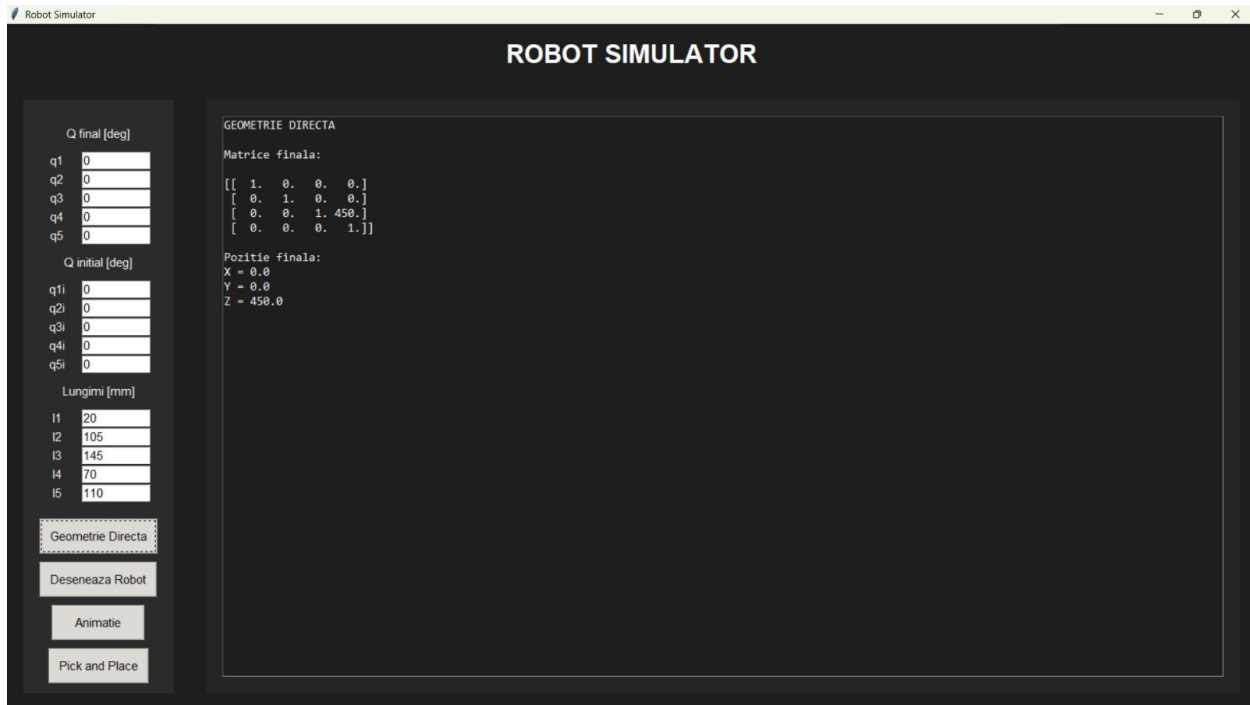
)
```

În funcția `creeaza_robot()` avem `return None` la final, pentru a evita alte erori în caz că robotul nu a fost creat.

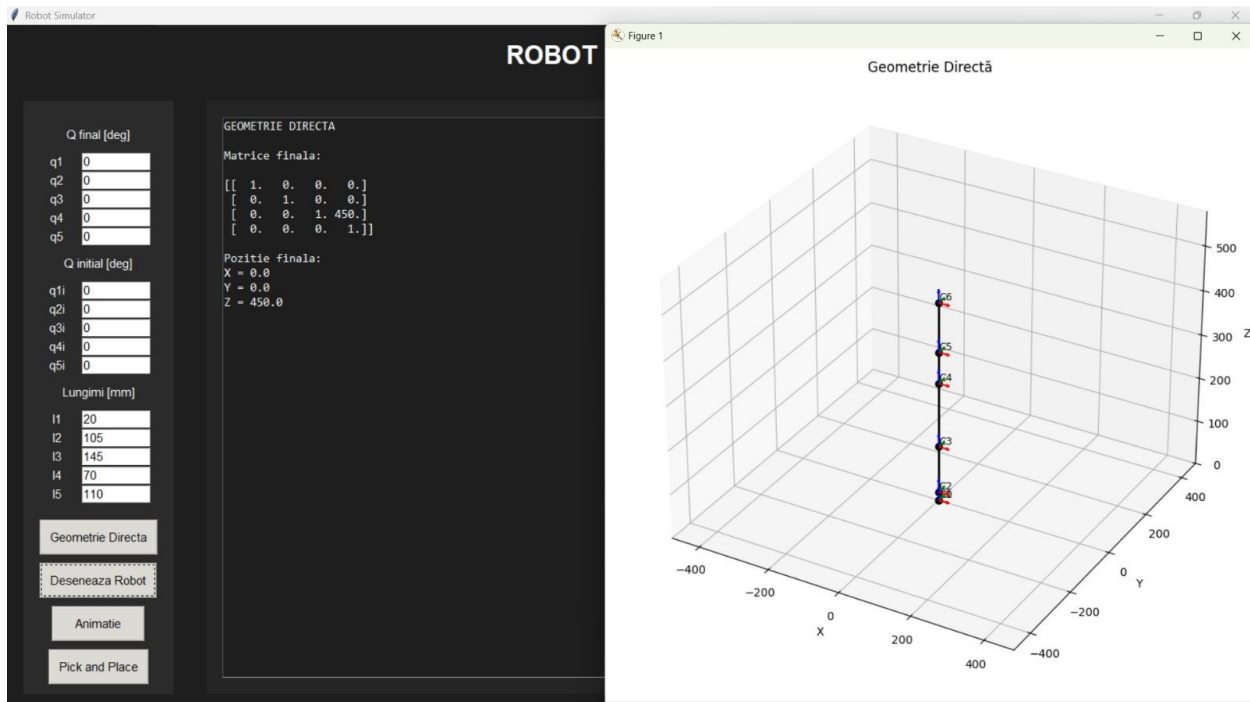
Interfața grafică realizată permite modificarea parametrilor cinematici variabili ai robotului, precum unghiurile articulațiilor și lungimile segmentelor, fără a necesita schimbarea codului sursă. Totuși, structura cinematică a robotului este definită intern în program, prin succesiunea transformărilor și tipul cuplelor utilizate în funcția de creare a robotului. Astfel, pentru implementarea unei alte configurații cinematice — de exemplu un robot cu alt număr de grade de libertate, alte axe de rotație — este necesară modificarea internă a codului aplicației. Interfața permite doar schimbarea parametrilor robotului existent, nu și redefinirea arhitecturii cinematice a acestuia.

Rezultate

În poziția 0 a robotului se poate observa faptul că suma lungimilor segmentelor cinematice conduce la aceeași poziție finală obținută prin calculul matricilor de transformare.



Desenul robotului în poziția 0:



Graficele poziție, viteză și accelerație pentru Q_final default.

ROBOT SIMULATOR

Q final [deg]

q1 180
q2 45
q3 50
q4 -60
q5 0

Q initial [deg]

q1i 0
q2i 0
q3i 0
q4i 0
q5i 0

Lungimi [mm]

l1 20
l2 105
l3 145
l4 70
l5 110

Geometrie Directa

Deseneaza Robot

Animatie

Pick and Place

GEOMETRIE DIRECTA

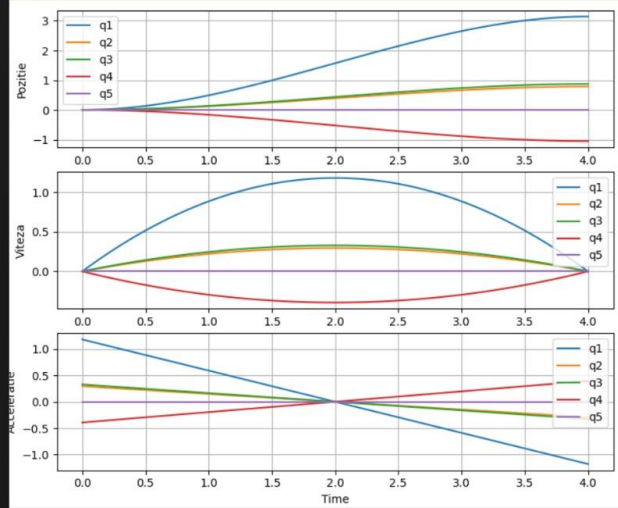
Matrice finala:

```
[[ -1.      0.      0.      0.      ]  
 [  0.    -0.81915204  0.57357644 321.93820178]  
 [  0.      0.57357644  0.81915204 229.05599731]  
 [  0.      0.      0.      1.      ]]
```

Pozitie finala:

X = 0.0
Y = 321.94
Z = 229.06

Figure 1



(x, y) = (0.863, -0.31)

Concluzii

În concluzie, aplicația dezvoltată demonstrează modul în care principiile programării orientate pe obiecte pot fi utilizate pentru modelarea și simularea sistemelor robotice într-un mod modular, clar și extensibil. Structura implementată permite definirea flexibilă a articulațiilor și mișcărilor robotului, precum și simularea funcționării acestuia în diferite configurații cinematice.

Funcționalitățile implementate, precum generarea transformărilor spațiale, reprezentarea geometrică a robotului și animațiile de tip Pick and Place, reflectă concepte utilizate frecvent în aplicațiile industriale moderne. Sisteme similare sunt întâlnite în companii din domeniul automatizărilor și producției industriale, unde roboții sunt utilizați pentru manipulare, asamblare, sortare sau transport automatizat al componentelor.